

SDP



From Issue to Pull Request

GitHub Copilot's Coding Agent in Practice

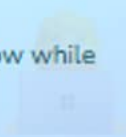
What if filing an issue was the same as fixing it?

"What if you could delegate an issue and get back a reviewed pull request?"

GitHub Copilot's coding agent handles the implementation loop —
leaving the hard decisions to humans, by design.

Developer

Delegate bounded tasks, stay in flow while agent builds PRs



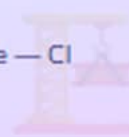
Tech Lead

Govern what the agent can touch with firewall and branch rules



Platform Engineer

Set up the environment agents use — CI parity from day one



12 min

total human time for a lodash upgrade across 23 files

95% detail

captured via image-based issue creation vs manual transcription

Platform-enforced

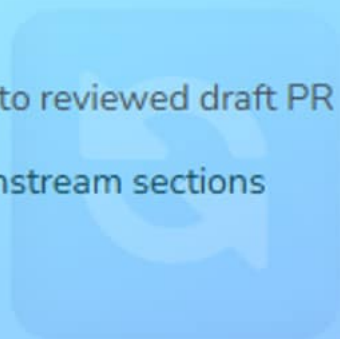
assigner cannot approve their own PR — by GitHub design

PART 1

Delegation Loop

The full loop from trigger to reviewed draft PR

Mental model for all downstream sections

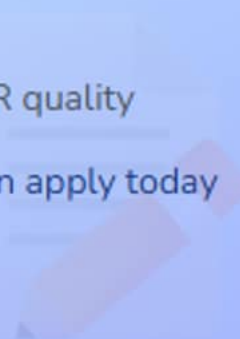


PART 2

Writing Issues

Structure issues to maximize PR quality

The most learnable skill you can apply today

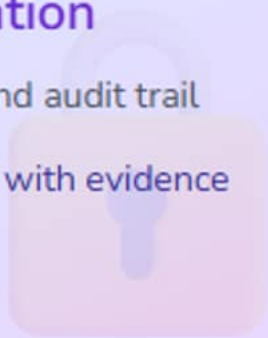


PART 3

Trust and Configuration

CI parity, firewall controls, and audit trail

Answer the safety objection with evidence

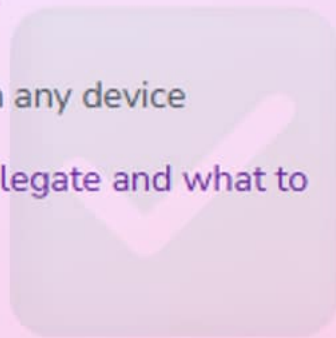


PART 4

Review Workflow

Evidence-first review from any device

Framework for what to delegate and what to keep



Click any section to jump directly there

Delegation Loop

Establish the full loop end-to-end: trigger, sandbox execution, evidence bundle, draft PR, and human review.



The Full Loop

Trigger to draft PR in one complete arc



Three Layers

Intelligence, Environment, Governance



Draft PR Boundary

Agent opens — platform enforces no self-merge

GitHub-enforced separation of duties
↳ **Assigner ≠ Approver** – by platform constraint

Three Layers Every Agent Session Uses

Intelligence, Environment, and Governance — the backbone all other sections reference

Intelligence

Model access, repo instructions, and agent skills — interface-agnostic

GPT-4.1 • Claude

Environment

Ephemeral GitHub Actions runner, fresh per session via setup-steps.yml

Ubuntu • CI parity

Governance

Agent Firewall, branch protection, and enforced separation of duties

Firewall • Reviews

The customization investment compounds — instructions and skills written for VS Code apply identically here

Direct Delegation vs. Configured Automation

Direct Delegation

Assign in browser, VS Code, or Mobile

One issue → one session, started immediately

No automation setup required — works on any repository

Best for ad-hoc bounded tasks and exploring the workflow

Configured Automation

Set a trigger comment text in Agents → Automations

Starts a new configured run — distinct from draft-PR refinement comments

Recurring handoffs: doc updates, investigations, follow-up issues

Admin must enable cloud-agent policy for Copilot Business / Enterprise

Choose Reasoning Level at Task Start



Default

Balanced quality and cost — right for routine tasks like dependency upgrades and test scaffolding



Higher Reasoning

Better on complex work — uses more tokens and more premium-request credits per run



Requirements

Supporting models only; requires a paid Copilot plan that includes the cloud agent

The Agent Can Never Merge Its Own PR

Platform-enforced, not policy — by design

2

required human reviewers before any agent change reaches the default branch

GitHub enforces: agent opens draft only · assigner cannot be approver

Draft status is invariant

Every coding agent session opens a draft PR — no configuration can change this

Assigner cannot approve

The person who assigned the issue is blocked from approving the resulting PR

Evidence bundle for reviewer 2

Agent reasoning, test results, and firewall alerts surface for the second reviewer

Plan for a second reviewer

Teams where leads both assign and review need a designated rotation approver

 This constraint makes every agent change structurally safer than a solo commit from a team member.

Writing Issues

PR quality is a direct function of issue quality — the most learnable and immediately applicable skill in this talk.



Issue Quality

Structured issues produce better PRs



Visual Workflow

Drop an image, get a complete issue



Acceptance Criteria

Context + criteria + constraints format

Image-based issue creation

↳ 14 min manual → 2 min visual · 95% detail capture

The Agent Interprets, Not Infers

✗ Vague Issues

Agent guesses at intent and scope boundaries

Ambiguous ownership

Agent may touch unrelated files

PRs arrive with wrong assumptions baked in

Rework cost often exceeds original implementation time

💡 Structured Issues

Acceptance criteria: testable, checkboxed conditions

Explicit scope boundaries

In-scope files and out-of-scope constraints listed

Context links: migration guides, related files, patterns to follow

✓ Consistent PRs

Agent follows spec — correctness is reviewable

Evidence bundle coherent with stated intent

12 min

review time on well-scoped issues

Same Task, Opposite Outcomes

Underspecified

Issue title: Fix the auth bug

Body: The auth is broken. Please fix it.

No acceptance criteria — done is undefined

No scope — agent may touch unrelated files

Structured

Issue title: Upgrade express-rate-limit v6 to v7

3 checkboxed, testable acceptance criteria

Scope: explicit in/out-of-scope file list

Context: migration guide + current file link

Drop a Screenshot — Get a Structured Issue

95% detail capture in 2 minutes vs. 14 minutes of manual transcription

2 min

to create a fully structured issue via image-based workflow

vs 14-minute manual transcription — 95% vs ~60% detail capture

Drop screenshot into github.com/copilot

Monitoring alerts, UI bugs, PagerDuty screenshots, error dialogs

AI extracts structured context

Error codes, timestamps, request IDs, stack traces, affected services

Template applied automatically

Labels assigned, severity assessed, issue ready for review in 2 minutes

Assign to Copilot to close the loop

Once the issue is confirmed, assign it to the coding agent for the PR



Operations teams report 9.3 hours/week of transcription time dropping to 1.3 hours/week.

Three Components That Maximize PR Quality



Acceptance Criteria

Testable, checkboxed conditions that define done — the agent runs these as a checklist

Specific and verifiable

One condition per checkbox



Scope Boundaries

Explicit in/out-of-scope prevents the agent from making helpful but unwanted changes

List specific files and dirs

Name what must not change



Context + Constraints

Migration guides, related PRs, patterns to follow, libraries or approaches to avoid

Link to relevant files

State performance requirements

Trust and Configuration

The credibility bridge: CI parity, allowlist firewall, and evidence-first auditability.



Setup Steps

Mirror your CI environment exactly



Agent Firewall

Allowlist-based outbound restriction



Evidence Logs

Blocked calls logged in the audit bundle

Blocked outbound call in evidence bundle

↳ Auditability is an artifact feature, not an audit step

Same Runtime as Your CI Pipeline

.github/workflows/copilot-setup-steps.yml

```
jobs:
  copilot-setup-steps: # required job name
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
```

🔑 Job name is the contract

copilot-setup-steps — only this name triggers; everything else is standard Actions syntax

🔒 Runs before firewall

Private installs and internal tools in setup steps need no allowlist entries

🔍 Debuggable in CI

Path trigger validates setup before the first agent session runs

Blocked Calls Appear in the Evidence Bundle

PR Evidence Bundle – Firewall Alert

 Firewall blocked outbound connection
Target: unauthorized-domain.com:443
Command: curl https://unauthorized-domain.com/upload
Time: 2026-04-06T14:32:17Z

Default allowlist

npm, PyPI, Docker Hub, GitHub APIs, and OS package managers — most repos need no configuration

Extend at org or repo level

Add internal registries and staging APIs under Settings → Copilot → Cloud agent

Blocked calls are audit artifacts

Any unauthorized outbound attempt logs here — investigate before merging any PR with alerts

Review Workflow

Close the loop: evidence-bundle review turns hours of skepticism into minutes of confidence.



Evidence Bundle

Changes + reasoning + blocked calls



Mobile Approval

Invoke @review-enforcer, approve in 3 min



What to Delegate

Bounded + well-understood → agent

Lodash upgrade across 23 files

↳ 12 minutes total human time – agent handled the rest

Review by Outcome, Not by Line Count

Without Evidence Bundle

- Open diff cold — unclear what changed or why
- Run tests locally to know if it works
30+ minutes before review can start
- No audit of external calls made during execution
- Agent reasoning is opaque — reviewer guesses intent

With Evidence Bundle

- Read summary first — 2–3 min for full change rationale
- Test results already visible
Pass/fail from agent CI run — no local run needed
- Firewall alerts surfaced — unauthorized calls visible immediately
- Invoke @review-enforcer for structured mobile analysis

<3 min

mobile review end-to-end

30 min

average PR wait time, down from 4 hours

2–3 min

to read evidence bundle and know what changed

When the Agent Wins vs. When to Stay Human

Take this framework back to your team tomorrow morning

Well-bounded	Approach is clear, scope is defined — dep upgrades, doc sync, lint	→ Delegate
--------------	--	------------

Design-only	Novel features, architecture decisions — keep human, use IDE Copilot	→ Human
-------------	--	---------

Multi-repo	Agent works one repo per session — cross-repo needs orchestration	→ Human
------------	---	---------

Local or DB	Production access, local DB, and local debug are outside the sandbox	→ Human
-------------	--	---------

Start with one delegation-candidate issue this sprint — bounded + well-understood is the only test

What Delegation Looks Like in Practice

A complete lodash upgrade, start to finish

12 min

total human time for a complete lodash upgrade
across 23 files

*Issue filed → agent executed → draft PR reviewed → approved after
one comment round*

Assign

Developer files structured issue: scope, acceptance criteria, migration guide link

Execute

Agent upgrades all 23 files, updates tests, opens draft PR with evidence bundle

Refine

Developer leaves 2 comments; agent pushes follow-up commits and re-runs CI

Approve

12 minutes total human time — agent handled the implementation

 The agent handled the rest. Human time = issue filing + PR review + one comment round.

From Hours of Oversight to Minutes of Review

Before

Issue transcription took 14+ minutes per task

Review required hours of skeptical, manual reading

No clear framework for what to delegate to AI

Audit was a separate step, not part of the artifact

After

Image drop creates a complete issue in under 2 minutes

Evidence bundle review completed on a phone in minutes

Decision tree: bounded + well-understood → delegate

Blocked firewall calls embedded in the evidence bundle

12 min

total human time for lodash upgrade
across 23 files

95%

detail captured in image-based issue
creation

<3 min

mobile PR review and approval end-to-
end

✓ What You Can Do Today

Today

- Assign one bounded, well-understood issue to the coding agent
- Use image-drop to create your next GitHub issue
- Review the evidence bundle on the resulting draft PR

This Week

- Add copilot-setup-steps.yml to mirror your CI toolchain
- Configure Agent Firewall to strict or moderate
- Build a structured issue template: context, criteria, constraints

This Month

- Audit agent PR patterns and refine your issue templates
- Enable Agents > Automations for repo-wide comment triggers
- Measure human-time-per-PR before and after delegation

🔑 Key Takeaway

The smallest delegation — one bounded issue — proves the loop and builds the confidence to expand it.

 Official Documentation[GitHub Copilot coding agent — VS Code](#)

Core concepts and triggering the agent from VS Code

[About the Copilot coding agent](#)

Architecture, security model, capabilities, and limitations

[Customizing the development environment](#)

copilot-setup-steps.yml reference and runner configuration

[Customizing or disabling the firewall](#)

Allowlist management, org-level controls, and firewall limitations

 Related Talks[Copilot CLI](#)

Terminal interface for triggering agent sessions from scripts and CI pipelines

[Agentic SDLC](#)

Multi-agent orchestration patterns for complex parallel

 Blog & Changelog[Assigning and completing issues with coding agent](#)

Delegation workflow, evidence bundle, and PR lifecycle

[Onboarding your AI peer programmer](#)

Issue quality, setup strategies, and adoption patterns

[Trigger Copilot automations with comments](#)

Repository-configured comment triggers and automations

[Customize the reasoning level for Copilot cloud agent](#)

Per-run reasoning controls and token tradeoffs



From Issue to Pull Request

GitHub Copilot's Coding Agent in Practice

12 min

total human time — lodash upgrade, 23 files, agent handled the rest

2 min

to create an issue with 95% detail capture via image-based workflow

Never

the agent can merge its own PR — platform-enforced separation of duties

What's the first bounded, well-understood issue you'd delegate to the coding agent?